ADMINISTRATION

HORIZON GRID

Administrator Guide

Version: 0.2.5

Documentation prepared: 2026-08-23

PREPARED FOR EVALUATION

Table of Contents

Administrator Guide

This guide is written for the one role that can see and change everything in HORIZON GRID: the **administrator**. It covers the parts of the platform that are either invisible to an Analyst or Viewer, or gated so that only an Administrator can act on them -- the first-run bootstrap account, creating and managing other users, the fixed role matrix, provider and AI configuration, provider health monitoring, the audit log, ports, database backup, and uninstall/reinstall.

This is deliberately a focused reference, not a repeat of the full User Manual. Where a topic is already covered in depth elsewhere in this documentation set, this guide gives you the administrator-relevant summary and points you to the fuller chapter rather than duplicating it: the Setup Wizard's page-by-page walkthrough lives in the First-Run Configuration chapter, the full 18-provider catalog and Manage Providers walkthrough lives in the Intelligence Providers chapter, and the full RBAC permission table with file-and-line citations lives in the Backend Security Architecture chapter. This guide assumes you already have an administrator account and are looking for how to operate the platform day to day, not how to install it for the first time.

1. The First Administrator Account

HORIZON GRID has no seed script, no hardcoded default account, and no "create admin" checkbox anywhere. Instead it has one simple, fixed rule: **the very first account ever registered on a fresh installation is automatically granted the ADMIN role.** The moment that first account exists, the public self-registration form closes itself -- every registration attempt after that is rejected outright (`HTTP 403` , "Self-registration is closed. Ask an administrator to create your account from the Administration page.") rather than quietly being handed a lesser role.

In practice you never call this endpoint directly. The Windows Setup Wizard's Administrator Account page collects your email, name, and password, and then registers exactly this account for you once the platform's containers report healthy -- by the time the wizard's final "Installation Complete" page appears, your administrator account already exists and works. The full page-by-page wizard walkthrough, including what a re-run ("reconfigure") of the wizard does differently once an admin account already exists, is in the First-Run Configuration chapter -- it is not repeated here.

The one fact worth stating plainly for an administrator specifically: **every account created after that first one must be created by an existing administrator**, from the Administration page's Users tab (below). There is no second bootstrap path, no database edit required, and no ceiling on how many administrator accounts can exist -- any admin can create another admin at any time.

2. The Administration Console

Once signed in as an administrator, the **Administration** link (under the Administration group in the top navigation) opens a four-tab console: **Overview**, **Users**, **Roles & Permissions**, and **Audit Log**.

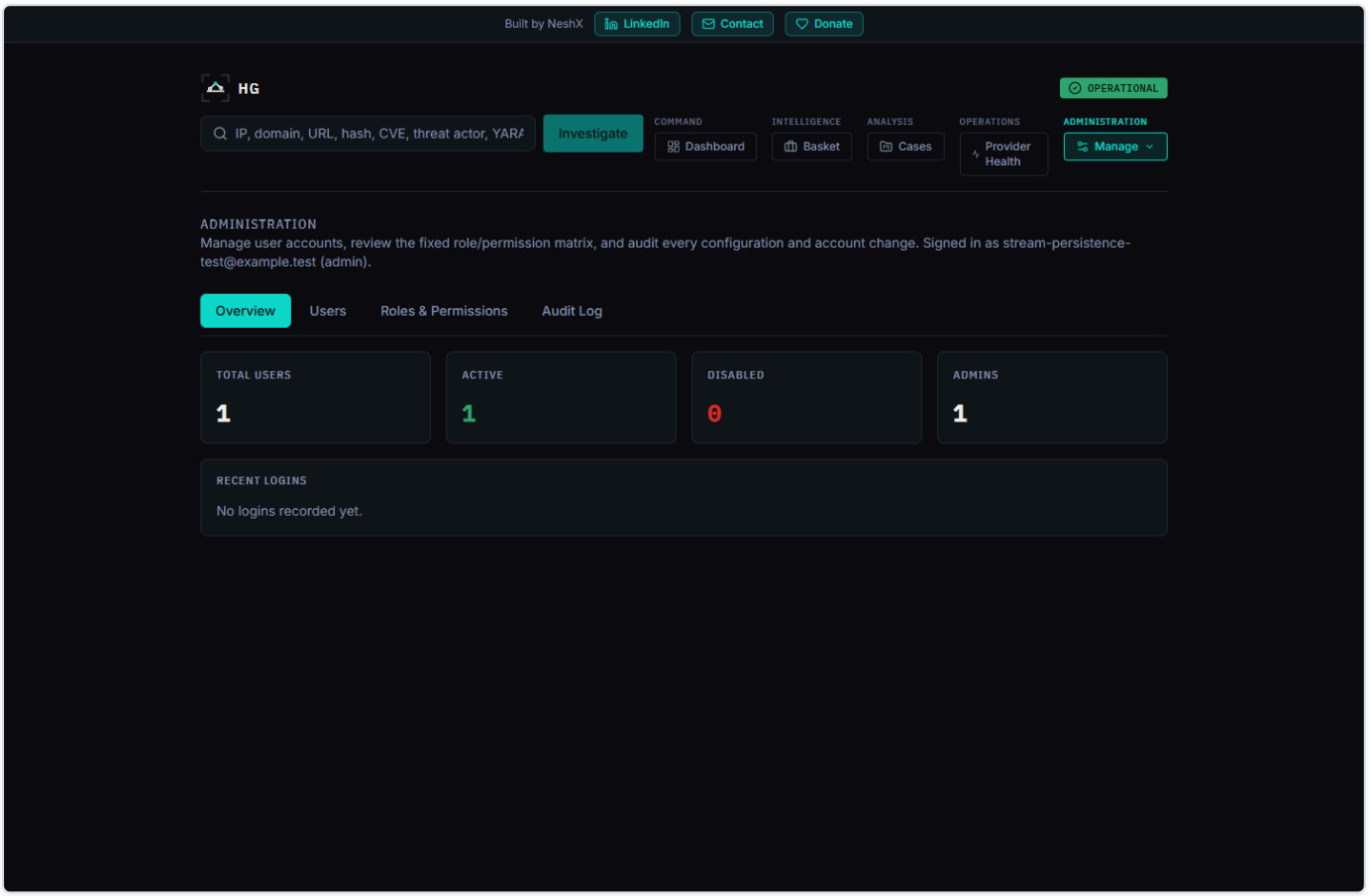

Figure 1 — The Administration console's Overview tab, showing total/active/disabled user counts, a per-role breakdown, and a list of recent logins.

Every one of this page's own admin-only checks (it redirects a non-admin visitor straight back to the dashboard) is a courtesy, not the real security boundary -- the actual enforcement happens on every single API call this page makes, server-side, via a `require_permission("user:manage")` dependency that re-derives your role from the database on every request rather than trusting anything embedded in your login token. Nothing here trusts the frontend to hide a button; a non-admin who somehow reached this page's buttons would still get a real `403` from the backend on every click.

The Overview tab is read-only: total users, how many are active vs. disabled, a count per role, and the five most recent logins across the whole platform.

3. Creating Users and Assigning Roles

The Users tab is where every account after the first one gets created. Click **+ New User** to open the create dialog: full name (optional), email (required, must be unique), an initial password (minimum 8 characters), and a role dropdown defaulting to Analyst. The dialog says plainly what's true of every account created here -- **the account is active immediately; there is no email verification step anywhere in this platform.** The new user can sign in with the password you just set the moment you click Create.

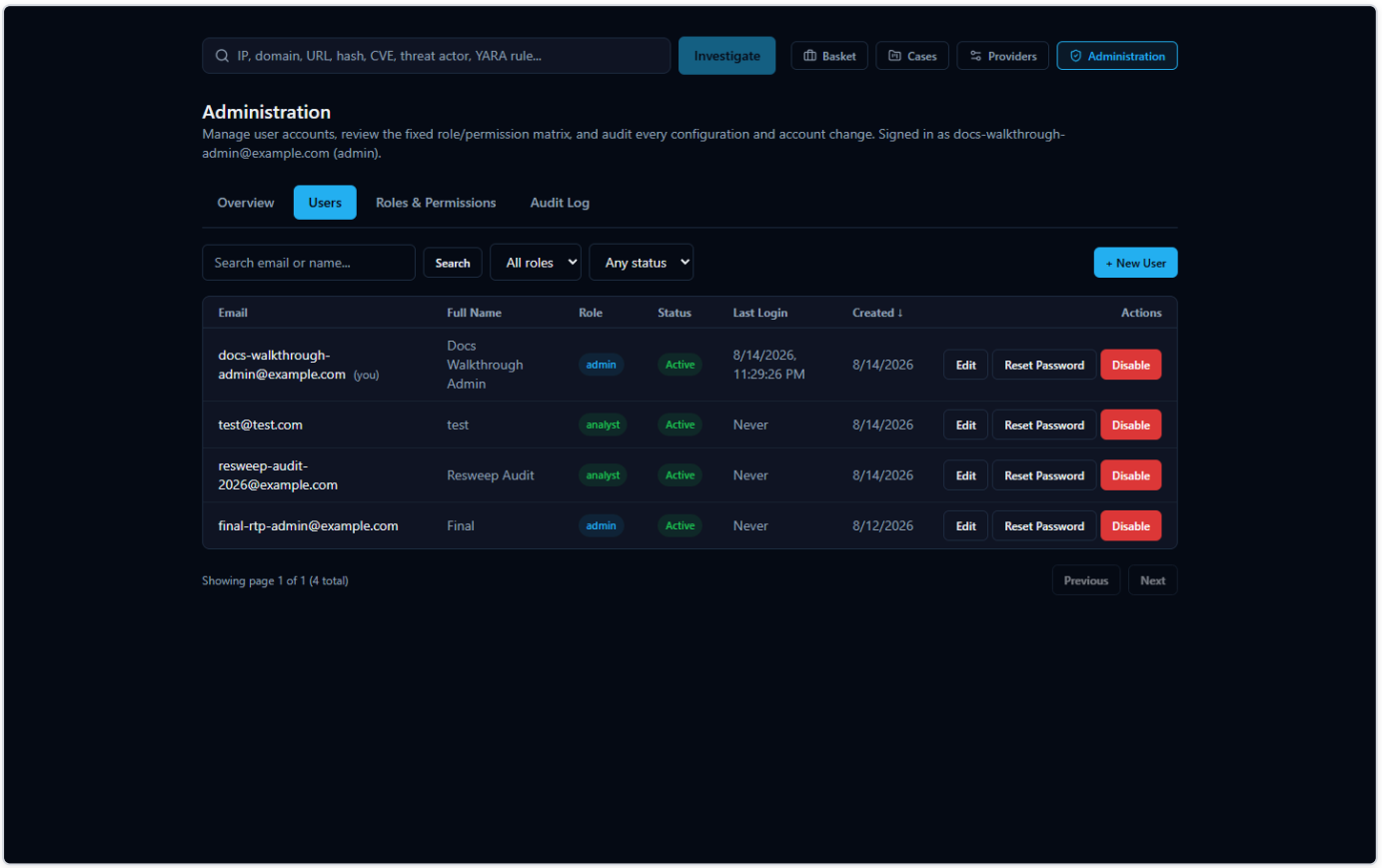

Figure 2 — The Create User dialog, with an email, initial password, and a role dropdown offering Admin, Analyst, or Viewer.

The Users table itself supports search (by email or name), filtering by role and by active/disabled status, column sorting, and pagination (25 rows per page) -- useful once you have more than a handful of accounts. Each row has three actions: **Edit** (name and role), **Reset Password**, and **Enable/Disable**.

3.1 What each role can and can't do

HORIZON GRID has exactly three roles, fixed in code (`Role.ADMIN` , `Role.ANALYST` , `Role.VIEWER`) -- there is no way to invent a fourth role or hand-tune a permission for one specific user. The Roles & Permissions tab of this same console (below) is a live, read-only view of the real permission matrix the backend actually enforces, so this table is guaranteed never to drift out of sync with reality:

Capability	Admin	Analyst	Viewer
Run a new IOC investigation	Yes	Yes	No
Read existing investigations, evidence, cases	Yes	Yes	Yes (read-only)
Export an investigation as PDF or CSV	Yes	Yes	No
Export an investigation as Markdown or JSON	Yes	Yes	Yes (client-side only, no permission check)
Generate AI analysis, hunting queries, use the Copilot	Yes	Yes	No
Manage the Basket; create/edit/close cases	Yes	Yes	No
Create/read Security Assessment Toolkit findings	Yes	Yes	Read only
View the Executive Dashboard and Provider Health	Yes	Yes	Yes
Configure AI backends and IOC providers	Yes	No	No
Create/edit/disable user accounts	Yes	No	No
Read the audit log	Yes	No	No

A few things about this table are worth calling out explicitly, because they're deliberate design decisions, not oversights:

- **Viewer's dashboard access is deliberately broad, not a mistake.** `dashboard:read` -- which gates the Executive Dashboard's KPIs and the Provider Health page -- is granted to all three roles on purpose. It only exposes read-only, operational-visibility data (queue depth, provider health, AI reliability); it never exposes a credential, a per-user field, or anything write-capable. A Viewer being able to see "is the platform healthy right now" without being able to touch anything is the intended shape of that role, not a gap.
- **Viewer genuinely cannot create investigations, export anything, or run a Security Assessment.** Its permission set contains no create/write/manage/generate permission anywhere in the matrix -- it is provably read-only, not just read-only by convention.
- **Only Admin can touch credentials or accounts.** `provider:manage` (AI backend and IOC provider configuration) and `user:manage` (everything in this console) belong to Admin alone. An Analyst can run investigations all day and never once be able to see or change a provider's API key.

3.2 Changes take effect immediately -- there is nothing to restart

Every protected route re-reads your role and your `is_active` flag from the database on every single request, rather than trusting the role embedded in your login token. That has a very concrete, practical consequence for you as an administrator: promoting someone to Admin, demoting an Admin to Analyst, or disabling an account entirely all take effect on that user's *very next request* -- not after they log out, not after a token expires, not after a restart. The Enable/Disable confirmation dialog says this outright: enabling "regains access immediately, with no restart required," and disabling means "any current session stops working on their very next request."

Password resets work slightly differently, because changing a stored password hash alone doesn't invalidate a JWT that was already issued under the old password. Resetting a user's password also increments an internal counter (`token_version`) that's checked against every access and refresh token that user presents -- so a reset immediately invalidates every session that user had open anywhere, not just future logins. The Reset Password dialog says this plainly: "This immediately signs the user out of every existing session -- they'll need to sign in again with the new password."

4. Guardrails: Protecting the Last Administrator

Two specific protections exist to stop an ordinary admin action from accidentally locking every administrator out of the platform.

You cannot change your own role. If you try to edit your own account in the Users tab, the role dropdown is disabled and the dialog tells you why: "You can't change your own role -- ask another administrator." This exists purely to prevent an accidental mid-session self-lockout (since a role change takes effect on your very next request, as above) -- it isn't a security boundary against a malicious admin, since any other admin can still change your role for you.

You cannot disable or demote the last active administrator. Attempting to disable the platform's only remaining active Admin, or to change that account's role away from Admin, is rejected outright ("Cannot disable the last active administrator." / "Cannot remove the ADMIN role from the last active administrator."). This check is deliberately race-safe, not just a simple count-then-act: the platform locks every Admin-role row in the database before counting how many would remain active, so two administrators simultaneously trying to disable two *different* remaining admins at the exact same moment cannot both succeed -- the second request is forced to wait for the first to finish, re-count against the now-current state, and correctly get rejected. A naive "count admins, then act" check without that lock could let exactly this kind of race leave the platform with zero administrators.

There is no hard delete for any user account, by necessity rather than by choice. Cases, evidence entries, and Basket items all reference the user who created them by a foreign key that can't be null -- deleting a user who has ever created a case, added evidence, or owned a basket item would either violate that constraint or require cascading away real investigation data. Disabling an account is the deletion mechanism here. This also has a side benefit worth knowing: it preserves the audit trail's own integrity, since every audit entry keeps a reference to the acting user too.

[MISSING FIGURE: admin-last-admin-protection-error.png]

5. Roles & Permissions Tab

This tab is a plain, read-only rendering of the exact same permission matrix the backend enforces on every request -- it exists so you can see, in one place, precisely what each role can and cannot do, without needing code access. It is not editable here: roles and permissions are fixed in code, not a configuration an administrator can hand-tune. If you need the full permission-string-to-route mapping (which exact API route each permission string gates), that level of detail lives in the Backend Security Architecture chapter and is not repeated here.

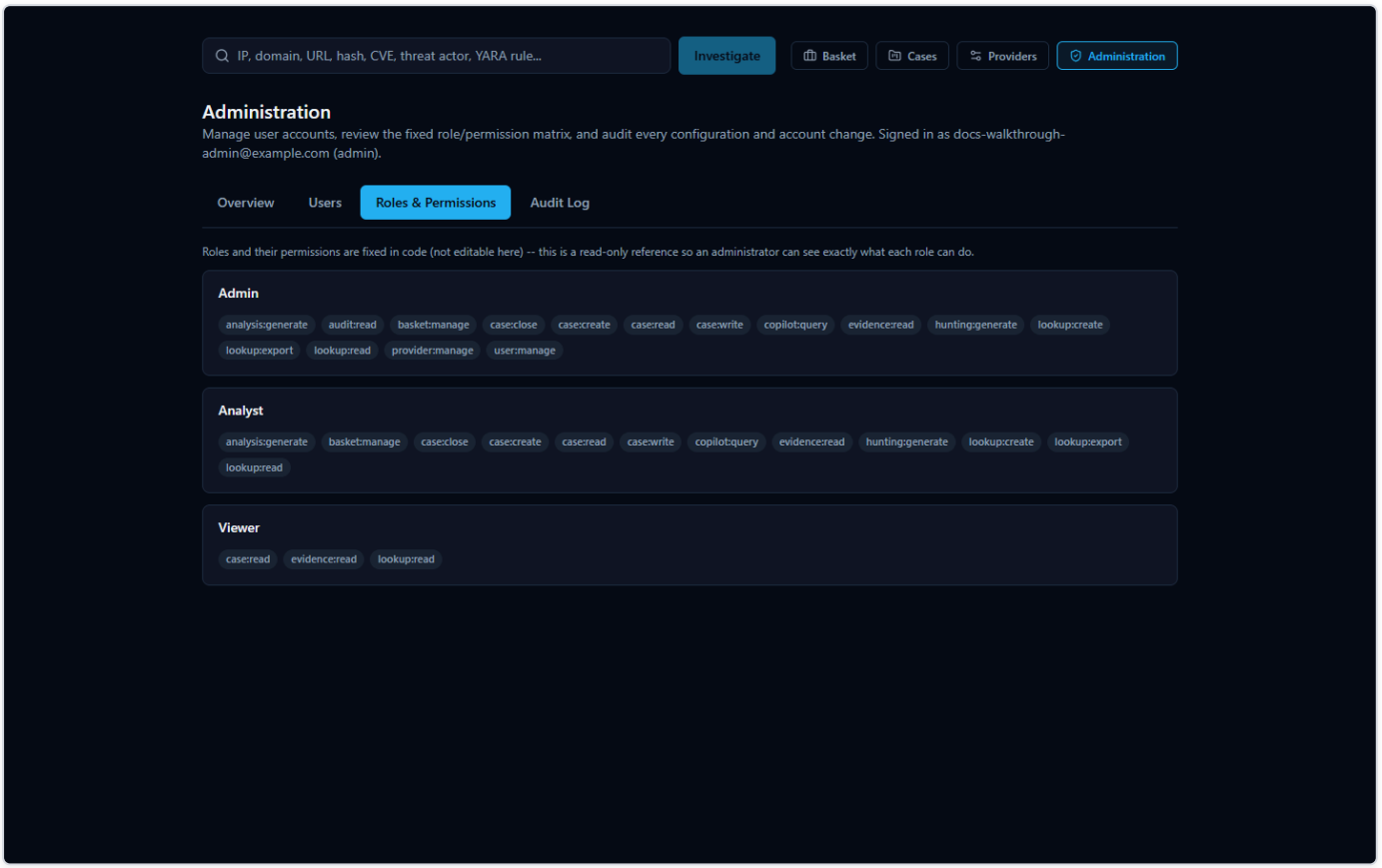

Figure 4 — The Roles & Permissions tab, showing each of the three fixed roles and its full list of permission strings as badges.

6. Provider Configuration

Provider and AI-backend configuration lives on a separate page from the Administration console: **Manage Providers**, reachable from the same Administration group in the navigation. It has two configuration tabs -- **AI Providers** and **IOC Providers** -- plus its own Audit Log tab and a Network Access tab (covered in Section 9). The full catalog of all 18 registered providers, what each one checks, and whether it needs a credential is covered in the Intelligence Providers chapter; this section covers only what's specifically relevant to an administrator operating this page.

Every change made here -- saving a credential, enabling or disabling a provider, switching the active AI backend -- takes effect on the very next investigation or AI call, with no restart. Every credential is Fernet-encrypted before it's ever written to the database, and every read path back out to the browser shows only a masked preview (dots followed by the last few real characters) -- there is no API response anywhere that returns a decrypted credential.

6.1 The Test button: a real quirk worth knowing about

The Test Connection button on both the AI Providers and IOC Providers tabs never falls back to an already-saved credential. If the field is blank, it tests blank -- you must retype the key before clicking Test, even for a provider you configured (and successfully tested) months ago.

This isn't a bug so much as a direct consequence of how the test endpoint is designed to work safely: `Test Connection` makes one real, live outbound call using *only* whatever value is currently typed into that field in your browser, and it deliberately never reads the stored, saved credential to do it -- not even to fall back on it if the field is empty. That's a real, meaningful safety property: the credential you're testing is always the exact one you're about to save, never some other value silently swapped in behind your back.

The practical consequence is the quirk above. Every masked credential field starts blank in the UI on page load -- the dots-and-last-four-characters preview you see is a placeholder hint shown *in* the field, not a real value sitting *in* the input. If you expand an already-configured provider's row and click Test Connection without typing anything first, you are testing an empty credential, which will report a failure (or, for some providers, `Not configured`) -- **not** a re-confirmation that the saved key still works. To actually re-test a saved key, retype it into the field first.

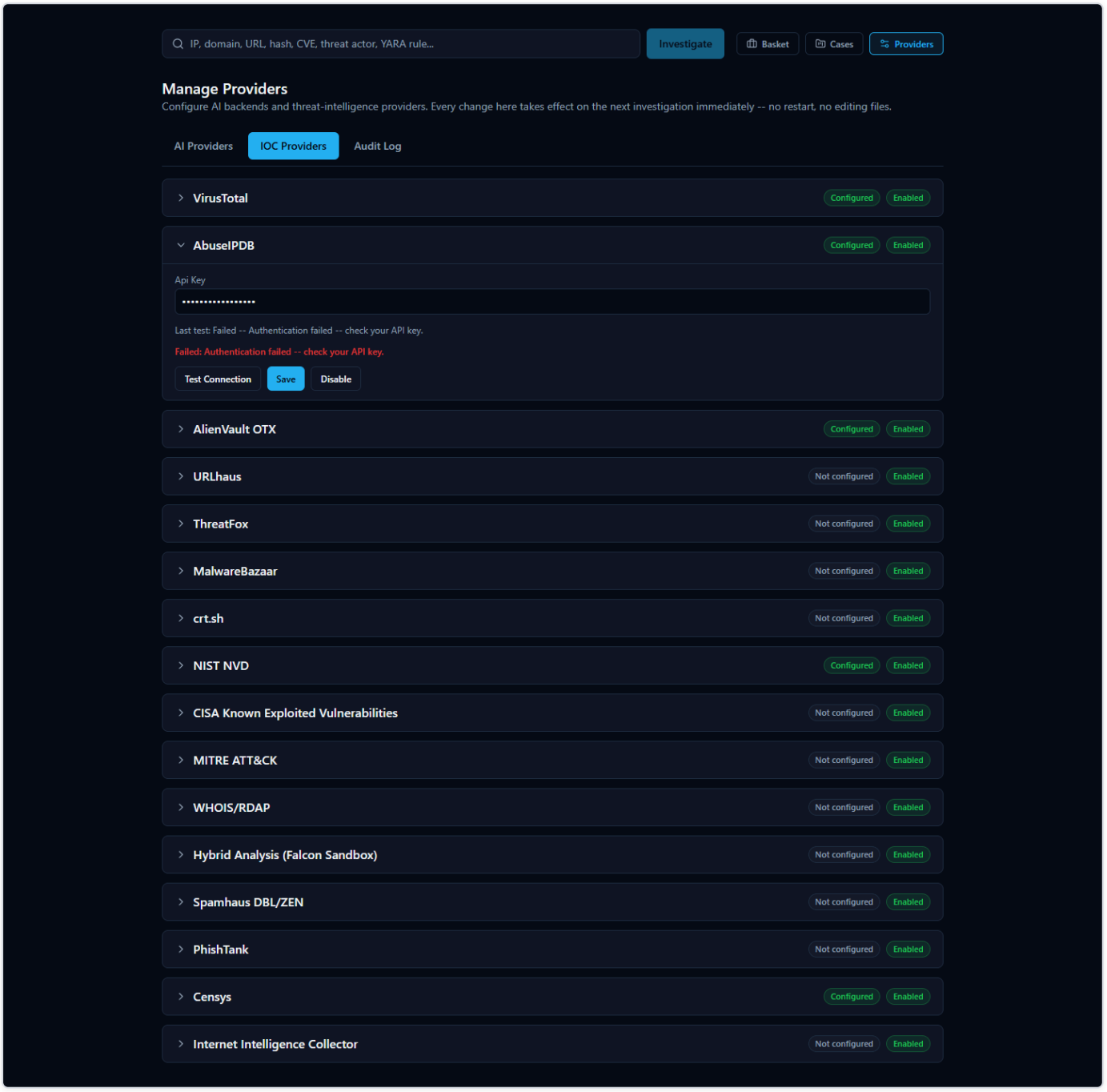

Figure 5 — An already-configured provider's row expanded, with the masked credential field still empty (showing only the "****...abcd"-style placeholder) and the Test Connection button about to be clicked -- clicking here without retyping the key tests a blank value, not the saved one.

One related behavior worth knowing so a routine "just confirm the model/notes field" save doesn't accidentally wipe a working key: saving a provider's configuration only ever changes the fields you actually retyped that session. If you open a provider's row purely to

change its model ID and click Save without touching the credential field, the existing saved credential is preserved untouched -- the save merges whatever you typed onto the already-stored values rather than overwriting the whole set.

6.2 Enabling, disabling, and setting the active AI backend

Each IOC provider row has its own Enable/Disable toggle, independent of whether it's configured -- a provider can be fully configured with a working key and still disabled platform-wide, which is useful if you want to temporarily stop using a source without losing its saved credential. AI backends work slightly differently: exactly one is "active" at a time (the one every new investigation's AI summary uses), switched with the **Set Active** button on that backend's row -- switching takes effect on the very next AI call made by anyone, with no restart.

7. Provider Health Monitoring

The dedicated **Provider Health** page (under the Operations group in the navigation, also linked from the Executive Dashboard's provider-health widget) is where you go to answer "is every individual provider actually working right now, or just configured" -- as opposed to the Manage Providers page, which only tells you whether a provider is configured and enabled. This page is intentionally visible to all three roles (Admin, Analyst, and Viewer all hold `dashboard:read`), since operational visibility into provider health is treated as broad, safe information rather than something to lock behind Admin.

The full mechanics of this page -- the four rolling time windows (1 hour/24 hours/7 days/30 days), the four status values (Healthy/Degraded/Down/Unknown), and the two real bugs found and fixed during this feature's own testing -- are covered in depth in the System Health and Troubleshooting chapter and are not repeated here in full. The one rule worth an administrator internalizing, because it directly shapes how you should read this page: **a provider with zero real attempts in a given window always shows "Unknown," never "Healthy,"** and a provider correctly reporting "nothing found" for an indicator counts as a healthy outcome, not a failure -- silence is never treated as evidence of health, and a provider doing its job honestly (most real lookups against most providers come back with nothing) is never mistaken for one that's broken.

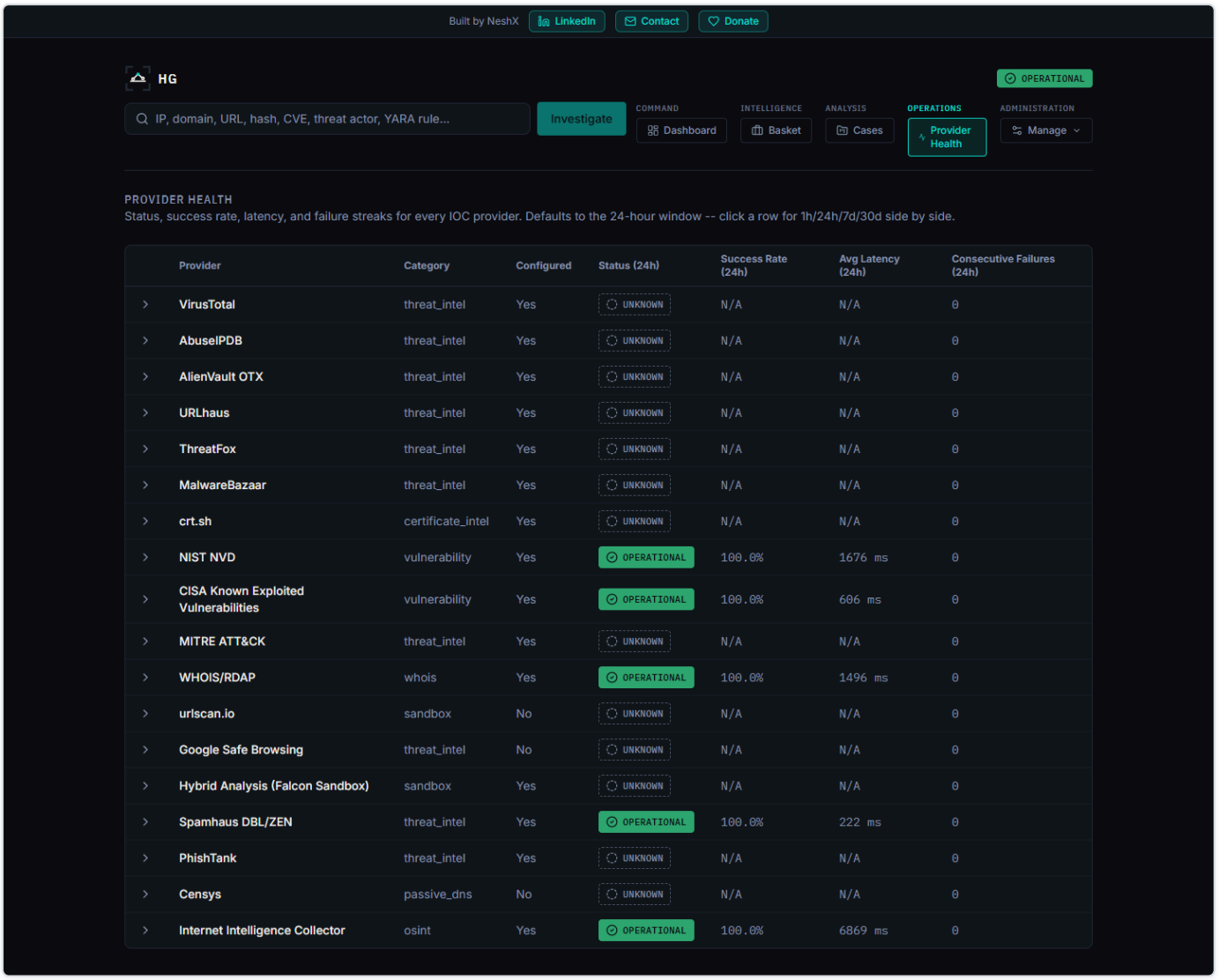

Figure 6 — The Provider Health page, showing per-provider status across four time windows with a distinct icon and label for Healthy, Degraded, Down, and Unknown.

8. Audit Log

Every configuration and account-management action in the platform funnels into one shared, append-only audit table -- the same table backs both the Administration console's Audit Log tab and the Manage Providers page's own Audit Log tab, since user-management events were deliberately routed into the *existing* audit sink rather than a second, parallel one. Reading it requires a dedicated `audit:read` permission, held by Admin only -- distinct from `provider:manage` and `user:manage`, so read access to the history is its own gate, not a side effect of being able to make changes.

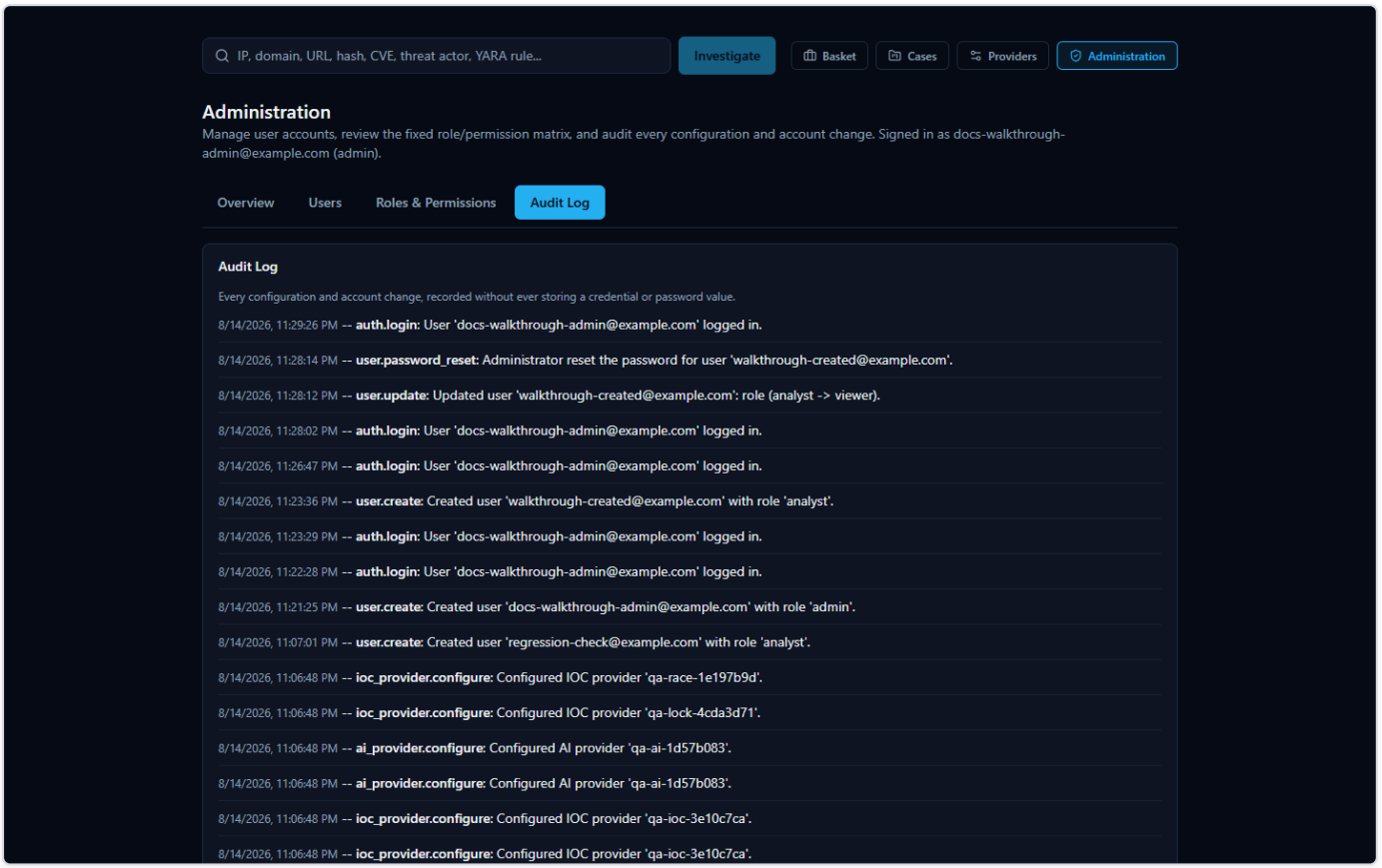

Figure 7 — The Administration console's Audit Log tab, showing timestamped entries with the acting administrator's email, the action, and a human-readable description.

What gets recorded: every AI-backend and IOC-provider configure/enable/disable/activate action, every connection-test result, the one-time migration event that seeds provider configuration from an existing installation's `.env` file, and every user-management and authentication event -- account creation, name/role updates, enable/disable, password resets, and both successful and failed login attempts. Each entry carries a timestamp, the action name, a human-readable description, and (for every action type except test-connection results and failed logins, where there is no authenticated actor to attribute it to) the acting administrator's own email address.

What is never recorded, on purpose: a credential, password, or token value. The description field that accompanies every entry is documented in code as human-readable text only -- every real call site writes a fixed, credential-free description (for example, "Configured AI provider 'anthropic'." or "Reset the password for user 'analyst@example.com'."), never an interpolated secret. The function that records a failed login doesn't even accept a password parameter in its own signature, so there is structurally no way for one to end up in that log entry by accident.

One honest, disclosed gap worth knowing about rather than assuming away: connection-test result entries (`ai_provider.test` , `ioc_provider.test`) record *what* happened and *when*, but not *who* clicked Test -- unlike every other action type in this log, those two don't currently carry an actor attribution.

9. System Settings and Network Ports

There is no separate in-app "System Settings" page beyond Manage Providers and the Administration console covered above. The one category of platform-wide setting that genuinely lives outside the web app is network ports (which port the web interface, backend API,

Postgres, Redis, Neo4j, and OpenSearch each bind to on the host machine) -- those are set on the Setup Wizard's Network Ports page, and changed the same way: by re-running the wizard from the **Configuration** shortcut in the Start Menu.

A given installation's actual port values are whatever was chosen (or accepted as the suggested default) on that page -- there's nothing universal to quote here, since the wizard lets you pick different values per install specifically to avoid conflicts with something else already running on the same machine. As one concrete example of what this looks like on a real, currently-running installation: the web interface on port 3000, the backend API on port 8000 (both reachable from other devices on the network), and Postgres, Redis, Neo4j, and OpenSearch all bound to the loopback interface only, on their own respective ports. Your installation may use different values; the platform runs identically either way.

The Manage Providers page's **Network Access** tab is where you go to *see* the current values without re-running the wizard -- it shows the address this computer answers on, the address other devices on the same network can use (detected automatically during setup), and the backend's port. This tab is read-only by design; to actually change a port, you re-run Configuration, not this tab. If your network address changes (for example, after a router restart), this tab will keep showing the stale, previously-detected address until you re-run Configuration to re-detect it -- there is no background poller that notices this on its own.

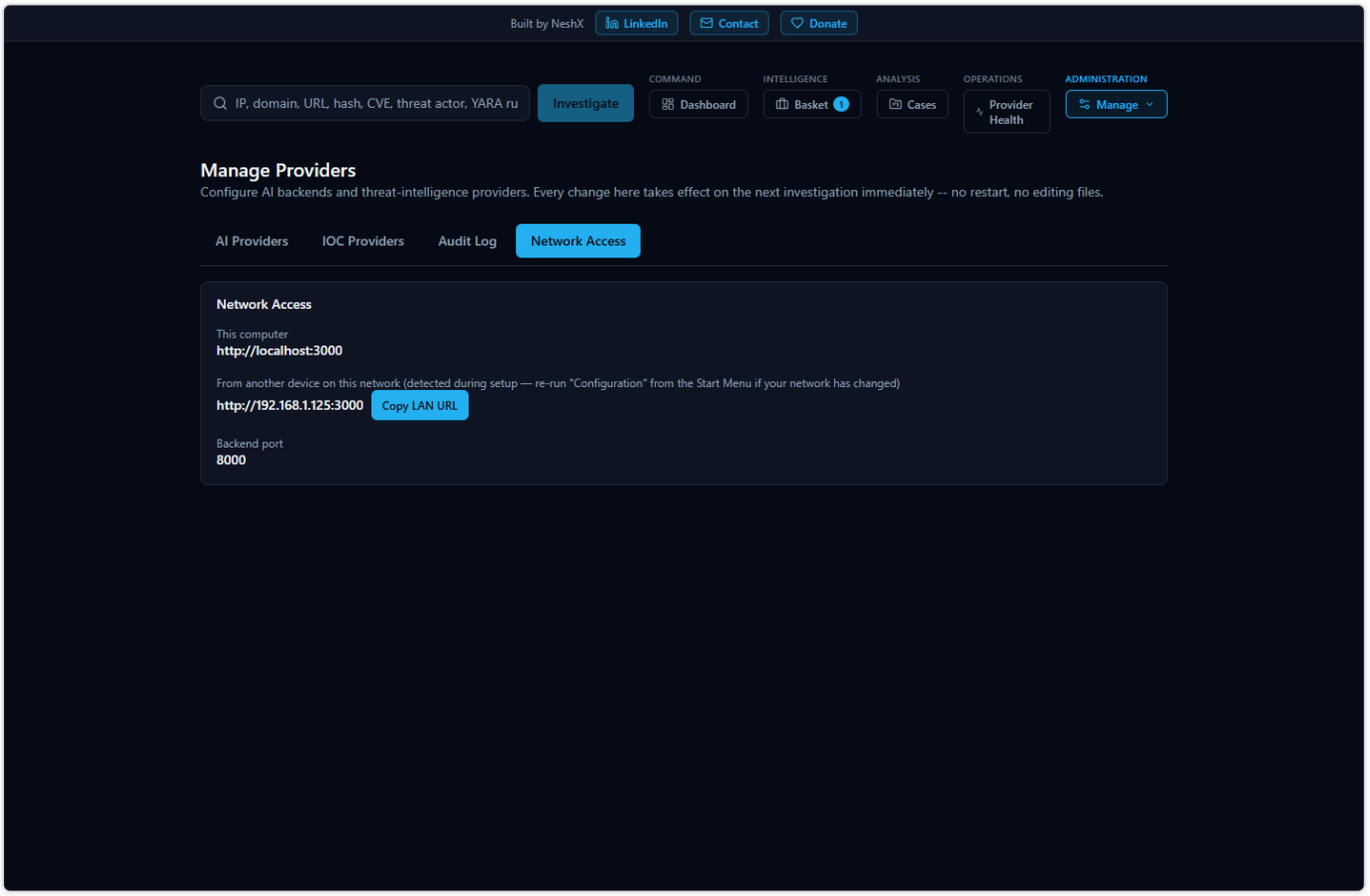

Figure 8 — The Network Access tab, showing the platform's local URL, the detected LAN URL for other devices, and the backend port -- with a note directing the administrator to the Configuration shortcut to change any of these.

Re-running the wizard on an already-installed platform pre-fills every page from the existing configuration (so it's a "review and adjust" flow, not a blank slate) and, before anything else changes, automatically takes a full database backup -- covered next.

10. Database Backup

HORIZON GRID backs up its Postgres database by running `pg_dump` *inside* the already-running Postgres container -- deliberately not requiring any separate Postgres client to be installed on the Windows host, since the one guaranteed to exist and match the server's exact version is the one already bundled in the same container image the platform itself runs.

This happens two ways:

- **Automatically, before every reconfigure.** Every time you re-run the Setup Wizard against an already-installed platform (the Configuration shortcut), it takes a backup first, before writing any new configuration or restarting anything.
- **On demand, from the Start Menu.** The **Backup Database Now** shortcut runs the exact same backup script by hand, any time you want one -- for example, right before you try something you're not 100% sure about.

Each backup lands in the platform's ProgramData backups folder as a timestamped `.sql` file, and only the 10 most recent backups are kept -- older ones are pruned automatically so this folder doesn't grow without bound across repeated upgrades and reconfigures. The backup script is written to fail safely and honestly rather than silently: if it finds no configuration yet, or finds the Postgres container isn't currently running, it says so and exits cleanly rather than producing a confusing error; if a backup attempt genuinely fails or produces an empty file, it says so plainly and removes the broken partial file rather than leaving a zero-byte dump behind that might be mistaken for a real one later.

[MISSING FIGURE: admin-backup-database-shortcut.png]

11. Uninstall and Reinstall

Uninstalling HORIZON GRID (via the **Uninstall** shortcut, or Windows's own "Add or remove programs") presents a real choice, not a single destructive default:

- **Remove Application (the default).** Removes the installed program files and stops the running containers, but deliberately leaves your configuration, credentials, and all investigation data -- cases, Baskets, notes, the database itself -- completely intact. Reinstalling later picks up exactly where you left off.
- **Remove Everything.** The explicit, destructive opt-in: this also deletes the Docker volumes (permanently destroying all case, investigation, and watchlist data) and deletes the entire ProgramData configuration directory. Because this is irreversible and there is no automatic backup taken before it, choosing this option requires typing `DELETE`, in capital letters, into a confirmation box -- a plain Yes/No isn't considered sufficient friction for an action this destructive.

Either path also removes the Windows Firewall rule the platform created for itself, scoped only to the Private network profile.

11.1 The password-mismatch install bug, and why you no longer need to think about it

A real, previously-reported failure existed in earlier builds of the installer: on a fresh install, the wizard always generates a brand-new random Postgres password. Postgres, however, only ever applies that password to a genuinely empty, never-before-initialized database directory -- once a database volume has been initialized once, every later container start ignores whatever password is in `.env` entirely and keeps using whatever password is already baked into that volume. If an earlier, abandoned install attempt on the same machine had left behind a Postgres volume (for example, an install that was started and then cancelled or interrupted before it ever ran a proper `docker compose down -v`), a fresh install's brand-new random password would never match that old volume's real one -- and the backend would crash-loop on a Postgres authentication error immediately after installation finished, with no obvious cause from the administrator's side.

This has been fixed, and fixed narrowly and specifically: **only** on a genuinely fresh install (no existing configuration was found to recover a real password from) does the installer now check for a leftover Postgres volume with no matching configuration, and remove it before generating a new password -- so the new password is always paired with a genuinely empty, freshly-initializing database, never an incompatible old one. This check is looked up by Docker Compose's own project/volume labels, not a hardcoded name, so it stays correct even if the underlying project structure changes. Critically, this fix is scoped so it can never touch a real, existing install: an upgrade or reconfigure of a platform that already has a working `.env` takes an entirely different code path that correctly preserves

and reuses its real existing password, and never reaches this cleanup logic at all. If you've hit this exact crash-loop symptom on an older build, a fresh install with the current installer resolves it automatically -- there is nothing you need to do manually to work around it.

12. Troubleshooting

This guide covers administration, not day-to-day symptom diagnosis. For "something looks wrong, what do I do" -- an AI backend that's unreachable, a provider card showing an unexpected status, a database outage mid-investigation, or how to read the Provider Health page's four status values in more depth -- see the System Health and Troubleshooting chapter, which is written specifically for that. For deployment-level and container-level symptoms (a service that won't start, a migration that didn't apply, a Docker Compose-specific error), see the Deployment and Troubleshooting chapter in the backend documentation set, which is keyed to real error strings and log output rather than the web UI.

Administering a Linux Install

This section maps the actions covered in the Windows Admin Guide to their real Linux equivalents. The underlying platform (Docker Compose, the FastAPI backend, the Next.js frontend, Postgres/Redis/Neo4j/OpenSearch) is identical on both platforms -- only the outer packaging/administration layer differs.

Admin actions: Windows vs. Linux

Windows Admin Guide action	Linux equivalent	Notes
Start Platform	`sudo horizon-grid start`	
Stop Platform	`sudo horizon-grid stop`	
Restart Platform	`sudo horizon-grid restart`	
Service Status	`sudo horizon-grid status`	
Open in browser (Start Menu shortcut)	`horizon-grid open`	Starts the platform if needed, then opens the browser; also reachable via the `horizon-grid.desktop` menu launcher (Exec= `horizon-grid open`)
Backup Database Now	`sudo horizon-grid backup`	Writes a `pg_dump` snapshot to `/var/lib/horizon-grid/backups/` ; resolves the real Postgres container via `docker compose ps -q postgres` (label/service-based), so it works correctly under any `COMPOSE_PROJECT_NAME` , not just the default
Diagnostics	`sudo horizon-grid diagnostics`	Bundles a redacted copy of `.env` (`KEY=***REDACTED***(set, N chars)` or `KEY=(empty)` -- real values never appear), same redaction behavior as Windows's `Diagnostics.ps1`
Configuration / Reconfigure Wizard	`sudo horizon-grid configure`	Terminal wizard (`linux/wizard/setup_wizard.py`) replacing the WinForms wizard; same step order: Welcome -> Administrator Account -> AI Configuration -> Threat Intelligence Providers -> Network Ports -> Summary
Check prerequisites	`sudo horizon-grid check`	No Windows Admin Guide analog; verifies Docker/Compose availability before setup
Uninstall	`sudo apt remove horizon-grid` / `sudo apt purge horizon-grid`	See below; `horizon-grid uninstall` itself only prints this guidance, it does not perform removal
Version	`horizon-grid version`	

On reconfigure of an existing install, re-entering the existing admin email+password signs in for the wizard session and enables live "Test Connection" calls (`POST /api/v1/providers/{id}/test` , `POST /api/v1/ai/test`) -- identical to Windows. On a fresh install, Test Connection cannot work yet at the provider/AI wizard pages because no backend is running at that point in the flow -- this is an application-level fact true on both platforms, not a Linux limitation.

File locations: Program Files/ProgramData vs. FHS paths

Windows	Linux	Contents
Program Files (app)	`/opt/horizon-grid/app/`	`backend/`, `frontend/`, `docker-compose.yml`, `docker-compose.prod.yml`, `docs/` -- read-mostly
ProgramData config	`/etc/horizon-grid/.env`	Real config/secrets; `root:root`, `chmod 600` (verified live)
ProgramData backups	`/var/lib/horizon-grid/backups/`	`pg_dump` snapshots
ProgramData logs\setup.log	`/var/log/horizon-grid/setup.log`	Setup/wizard log
Start Menu shortcut	`/usr/bin/horizon-grid` + `/usr/share/applications/horizon-grid.desktop`	Single CLI entrypoint; desktop menu launcher runs `horizon-grid open` (no custom icon yet -- falls back to a generic one)
Windows Service registration	`/lib/systemd/system/horizon-grid.service`	Registered (`daemon-reload`'d) at install time but **not** auto-enabled or started -- nothing starts until you run `horizon-grid configure`, matching the Windows installer's own "nothing auto-starts before configuration" behavior

Both platforms' Compose project directory shares the basename `app`, so both produce the same Docker Compose project label (`com.docker.compose.project=app`) by design -- useful to know if you're correlating `docker compose ps` output across platforms.

Uninstall model: remove vs. purge

Linux uses two distinct `apt` verbs, mirroring the Windows uninstaller's own keep-data/remove-everything choice:

- **`sudo apt remove horizon-grid`** -- the direct analog of choosing "keep my data" in the Windows uninstaller. Keeps `/etc/horizon-grid`, `/var/lib/horizon-grid`, and all Docker volumes (Postgres/Neo4j/OpenSearch data). Containers stop, but volumes and config survive; reinstalling later picks up right where you left off. Confirmed live in both the three-distro test suite (33/35 checks passing per distro, both non-blocking exceptions explained in the Linux QA report) and, separately, under the real default project name inside an isolated test environment: containers stopped, volumes/config intact.
- **`sudo apt purge horizon-grid`** -- the direct analog of choosing "remove everything" in the Windows uninstaller. Deletes everything: stops and removes every container and volume labeled for the Compose project, and deletes `/etc/horizon-grid`, `/var/lib/horizon-grid`, and `/var/log/horizon-grid`. No undo.

Cleanup is **label-based** (`com.docker.compose.project=<project>`), not a hardcoded container-name guess, so it works correctly even if you've customized `COMPOSE_PROJECT_NAME`. A dedicated isolated test using the real default project name (`app`) confirmed this logic fully: 12/12 checks passed, including "all containers removed" and "all volumes removed." The full three-distro suite reported the same purge/remove behavior with two non-blocking failures per distro that were specific to that test harness's own project-naming choice (used only to avoid colliding with another instance on the shared test host) -- see **HORIZON_GRID_LINUX_QA_REPORT.pdf** for the complete, unfiltered results.

One packaging fix worth knowing about: earlier builds of `apt purge`/`apt remove` could leave one small untracked file behind (`/opt/horizon-grid/app/.env`, a runtime-written copy dpkg's manifest never tracked), which blocked full removal of that directory. This is fixed in the package's `postrm` script -- the Linux analog (via dpkg not tracking the file, rather than an ACL block) of a documented Windows uninstaller fix for the same underlying problem.

Firewall handling

Windows Firewall is always present and gets a guaranteed rule from the installer. Linux has no equivalent guarantee -- `horizon-grid configure` and the CLI make a best-effort detection of `ufw` or `firewalld` if active and configure them accordingly, clearly reporting either what it configured or that no firewall manager was detected, rather than assuming one exists.